МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №1

Тема: **«Порождающие паттерны: Одиночка, Строитель»**

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: реализовать решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов «Строитель» или «Одиночка».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Одиночка

Находится в coin_collector/settings.py, классы SettingsManager и GameSettings.

Задача: настройки звуковых эффектов и сложности должны быть общими для меню и игрового процесса. Если бы каждая часть приложения создавала собственный менеджер, значения могли бы разойтись: меню изменило бы одну копию, а игра продолжила бы использовать другую.

Как работает: метод __new__ сохраняет созданный экземпляр в атрибуте класса и при последующих вызовах возвращает тот же объект. Флаг _initialized также не позволяет повторно загружать настройки при получении этого объекта.

```
class SettingsManager:
    _instance: SettingsManager | None = None

    def __new__(cls) -> SettingsManager:
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self) -> None:
        if getattr(self, "_initialized", False):
            return
        self.settings = self._load()
        self._initialized = True
```

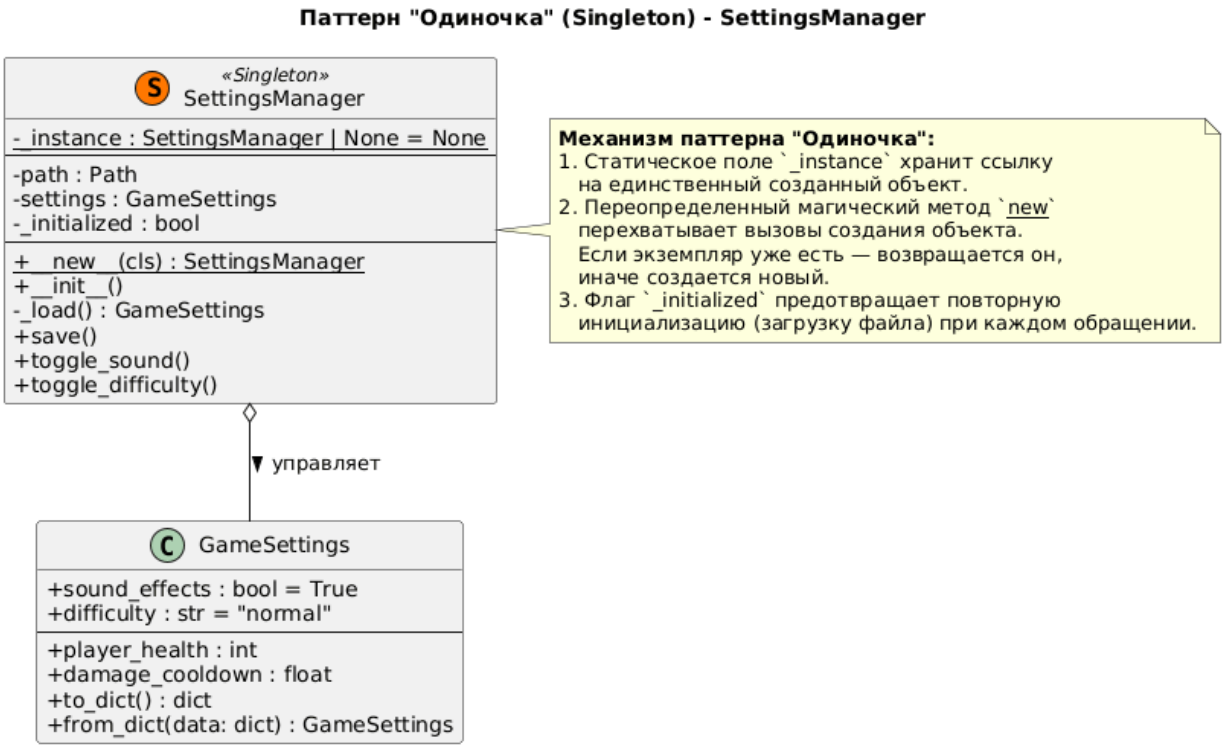


Рисунок 1. Диаграмма класса SettingsManager

Таким образом, настройки представляют общее состояние одного приложения, и отдельное хранилище для каждого экрана не нужно.

2. Строитель (Builder)

Находится в coin_collector/level.py, класс GameLevelBuilder.

Задача: собрать арену из начальной позиции игрока, монет и опасностей, не перегружая конструктор уровня длинным списком позиционных аргументов.

Как работает: строитель хранит промежуточное описание уровня. Вызовы set_player_start(), add_coin() и add_hazard() добавляют в описание отдельные части. build() проверяет координаты и создаёт готовый объект GameLevel.

```python
factory = ArcadeObjectFactory()
size = (960, 640)

level = (
    GameLevelBuilder(factory, size)
    .set_player_start((480, 360))
    .add_coin((120, 150))
    .add_coin((840, 150))
    .add_hazard((480, 220))
    .build()
)

print(len(level.coins))    # 2
print(len(level.hazards))  # 1
```

Строитель отклоняет некорректные координаты до создания объектов:

```python
builder = GameLevelBuilder(factory, size)
builder.add_coin((0, 0))  # за границами игровой арены
builder.build()           # ValueError: координаты объектов должны быть внутри
```

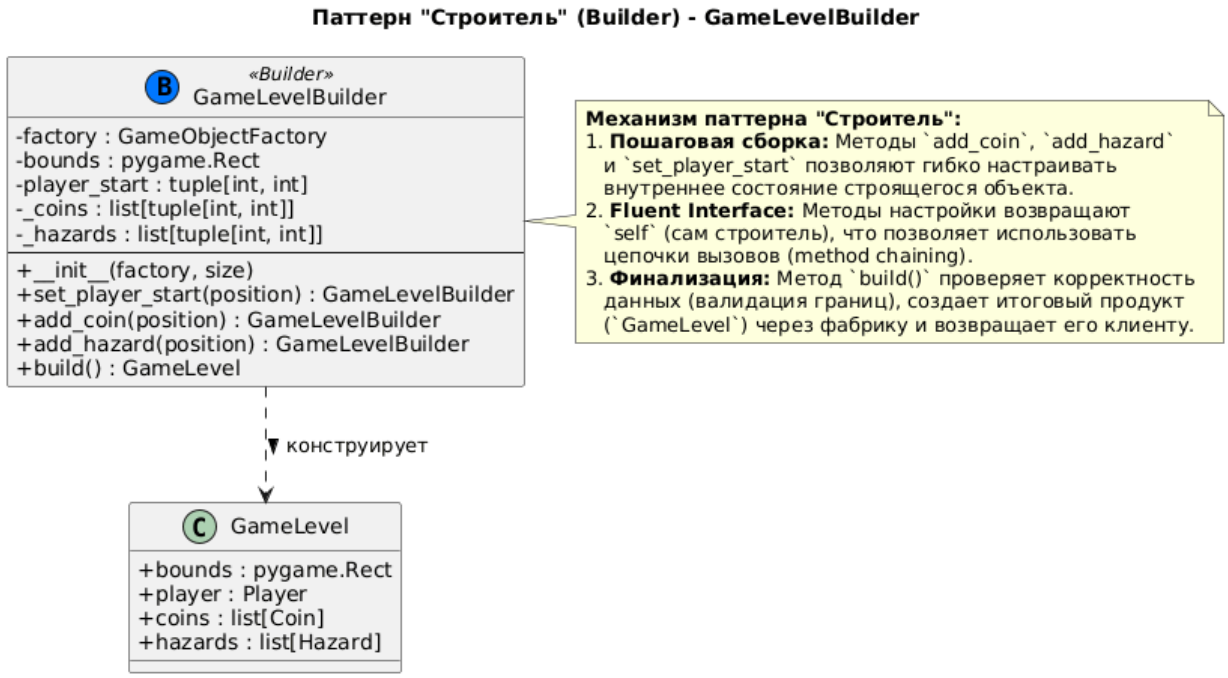


Рисунок 2. Диаграмма класса GameLevelBuilder

Таким образом, состав и размещение объектов уровня можно задавать по шагам, а результат сборки проверять до начала игры.

Вывод:

Проделав работу, реализовал решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов «Строитель» и «Одиночка».